



Programación Orientada a Objetos 2

Pruebas Unitarias

Creado por:

Julio César Méndez Sandigo

Docente:

José Durán García

Managua, 09 de Junio 2026

Introducción

En la presente práctica se desarrolló una aplicación Android sencilla utilizando el lenguaje Kotlin y Jetpack Compose para la construcción de la interfaz gráfica. La aplicación consiste en una calculadora básica que permite realizar operaciones simples y mostrar el resultado en pantalla mediante botones.

El propósito principal de esta actividad fue comprender la importancia de las pruebas automatizadas dentro del desarrollo de aplicaciones móviles. Para ello, se implementaron pruebas unitarias destinadas a validar la lógica de negocio de la aplicación y pruebas de interfaz para comprobar el comportamiento visual e interactivo de los componentes creados con Jetpack Compose.

Mediante esta práctica se pudo observar la diferencia entre probar directamente una función del código y probar la interacción del usuario con la interfaz gráfica. Esto permite detectar errores antes de publicar o entregar una aplicación, mejorando la calidad y confiabilidad del software.

Objetivo general

Implementar y ejecutar pruebas unitarias y pruebas básicas de interfaz en una aplicación Android desarrollada con Jetpack Compose, con el fin de verificar el correcto funcionamiento de la lógica de negocio y de los componentes visuales.

Objetivos específicos

- Crear una aplicación Android utilizando Kotlin y Jetpack Compose.
- Implementar una clase de lógica llamada Calculadora.
- Diseñar una interfaz sencilla con botones y un texto de resultado.
- Crear pruebas unitarias para validar operaciones matemáticas.
- Crear pruebas de interfaz para validar el comportamiento de los botones.
- Analizar la diferencia entre pruebas unitarias y pruebas de interfaz.
- Documentar el proceso mediante capturas de pantalla y explicación escrita.

Desarrollo de la aplicación

Para iniciar la práctica se creó un nuevo proyecto en Android Studio utilizando la plantilla Empty Activity, el lenguaje Kotlin y Jetpack Compose como herramienta para diseñar la interfaz gráfica.

La aplicación desarrollada contiene una clase llamada Calculadora, la cual se encarga de realizar operaciones matemáticas básicas. Inicialmente se implementó una función para sumar dos números, y posteriormente se agregaron funciones complementarias para restar, multiplicar y dividir.

La interfaz de usuario fue diseñada mediante una función composable llamada PantallaCalculadora. Esta pantalla contiene un botón principal llamado Calcular, que realiza la suma de $5 + 3$ y muestra como resultado el valor 8. Además, como parte de las actividades complementarias, se agregó un segundo botón llamado Restar, que realiza la operación $5 - 3$ y muestra como resultado el valor 2.

El resultado se muestra en un componente Text identificado mediante un testTag llamado resultado. Esta etiqueta permite que la prueba de interfaz pueda localizar el texto y verificar si muestra el valor esperado después de presionar un botón.

Código principal implementado

La clase Calculadora contiene los métodos necesarios para realizar las operaciones matemáticas básicas: suma, resta, multiplicación y división. Estas funciones representan la lógica de negocio de la aplicación, ya que son independientes de la interfaz gráfica.

La pantalla principal de la aplicación fue implementada con Jetpack Compose. En ella se agregaron botones para realizar operaciones y un texto donde se muestra el resultado obtenido.

El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

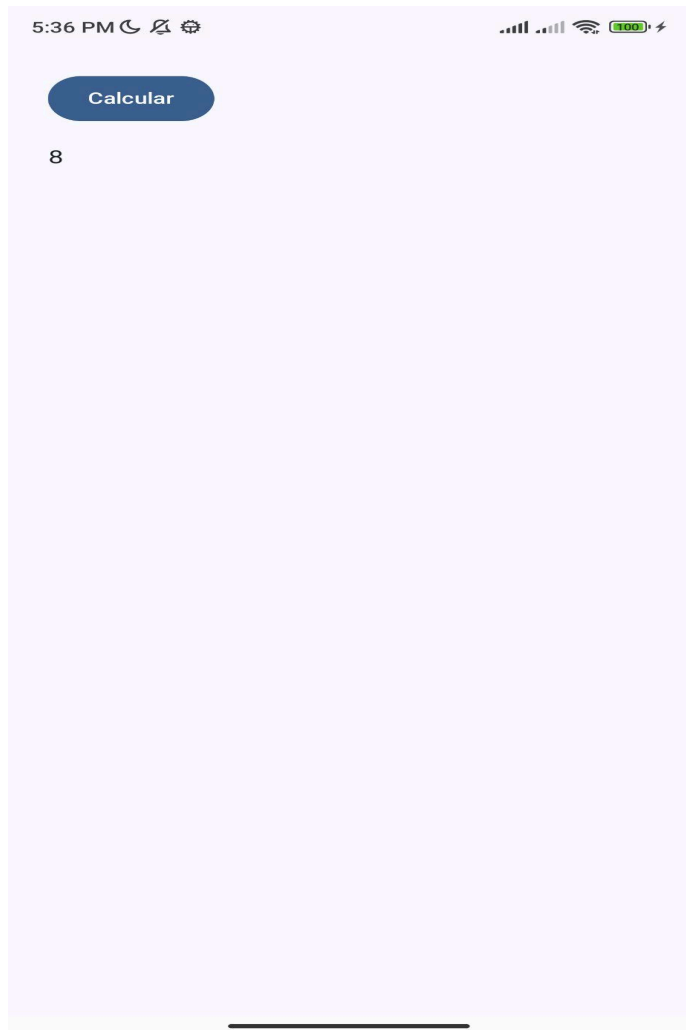
https://github.com/JulioMS-DEV/Pruebas_Unitarias.git

En el repositorio se incluyen los archivos principales de la aplicación Android, la clase Calculadora, la interfaz desarrollada con Jetpack Compose, las pruebas unitarias y las pruebas de interfaz implementadas para validar el funcionamiento del proyecto.

Ejecución de la aplicación

Después de implementar la lógica y la interfaz, se ejecutó la aplicación en un emulador de Android. Al presionar el botón Calcular, la aplicación mostró el resultado 8, correspondiente a la suma de $5 + 3$.

Evidencia de la aplicación funcionando con el botón Calcular:



Como parte de las actividades complementarias, también se agregó el botón Restar. Al presionar este botón, la aplicación mostró el resultado 2, correspondiente a la operación $5 - 3$.

Evidencia de la aplicación funcionando con el botón Restar:

6:26 PM



Calcular

Restar

2

Pruebas unitarias

Las pruebas unitarias fueron creadas en la carpeta test del proyecto. Estas pruebas permiten validar el funcionamiento de la lógica interna de la aplicación sin necesidad de ejecutar la interfaz gráfica ni utilizar un emulador.

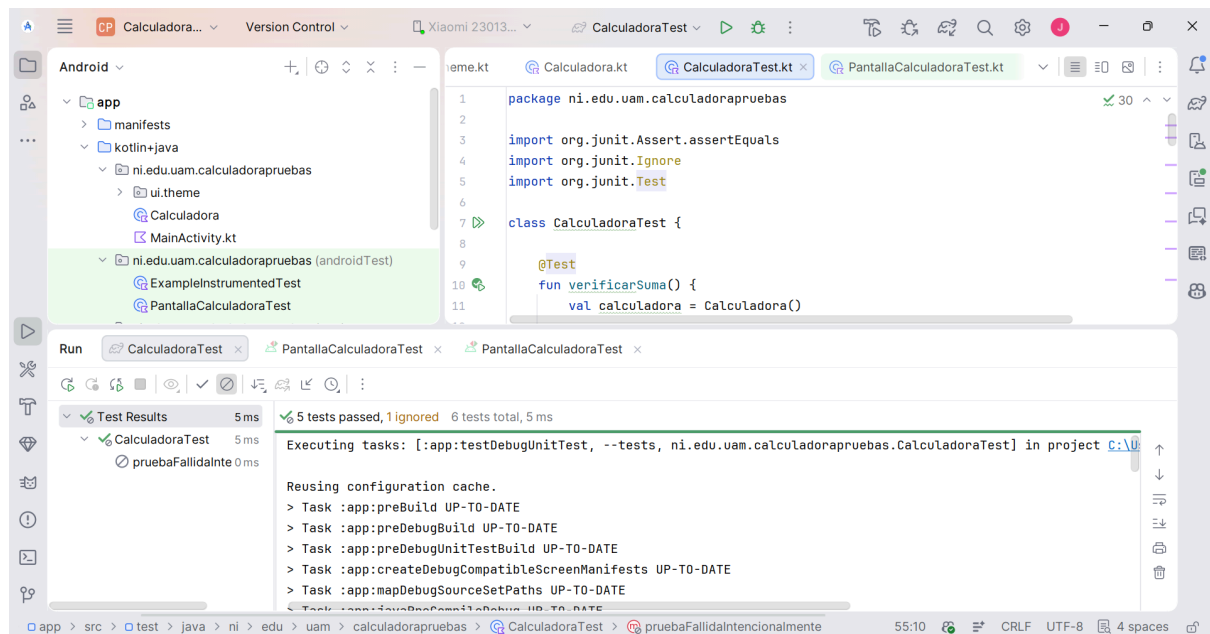
En esta práctica se creó la clase CalculadoraTest, donde se probaron las operaciones matemáticas implementadas en la clase Calculadora. La primera prueba validó que la suma de $5 + 3$ devolviera correctamente el resultado 8.

Posteriormente, como parte de las actividades complementarias, se agregaron pruebas para validar la resta, la multiplicación, la división y el comportamiento con números negativos.

Las pruebas unitarias implementadas validan lo siguiente:

- Que la suma de $5 + 3$ devuelva 8.
- Que la resta de $10 - 4$ devuelva 6.
- Que la multiplicación de 5×3 devuelva 15.
- Que la división de $10 / 2$ devuelva 5.
- Que la suma de $-5 + -3$ devuelva -8.
- Que la resta de $-5 - (-3)$ devuelva -2.

Evidencia de pruebas unitarias ejecutadas correctamente:



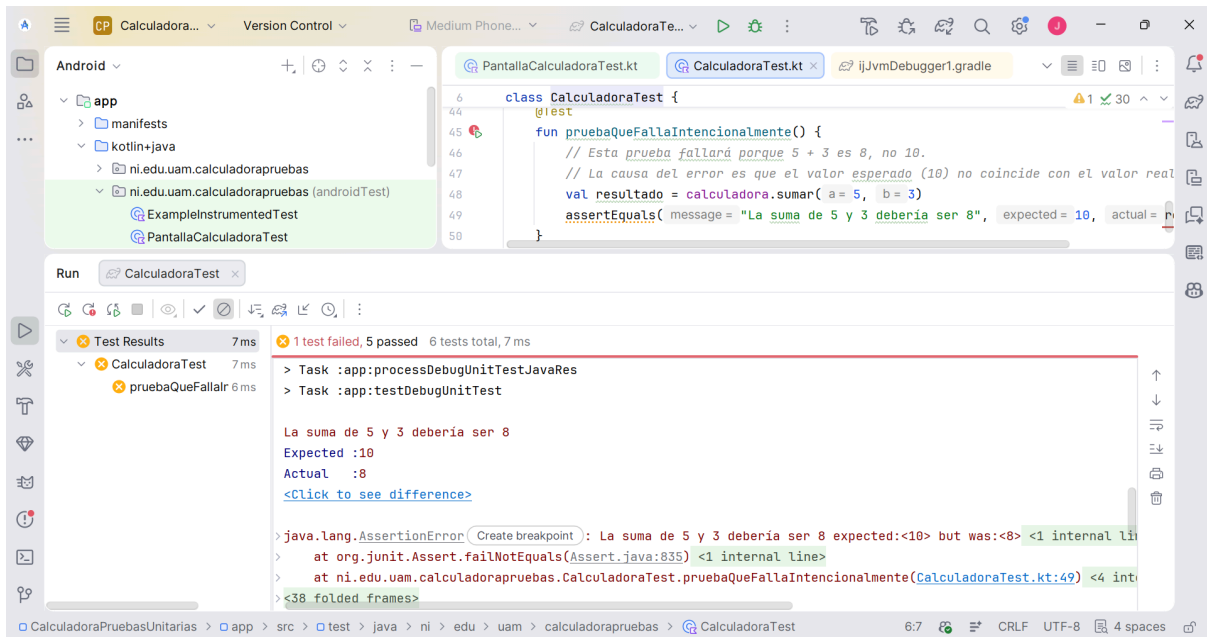
Prueba fallida intencionalmente

Como parte de las actividades complementarias, se diseñó una prueba que falla de forma intencional. Esta prueba consiste en sumar $5 + 3$, operación cuyo resultado correcto es 8, pero dentro de la prueba se colocó como resultado esperado el valor 10.

La prueba falló porque el valor esperado no coincide con el valor real devuelto por la función. Este ejemplo permite comprender cómo las pruebas automatizadas detectan errores cuando el resultado obtenido no es igual al resultado esperado.

En este caso, el error no está en la función sumar, sino en la prueba, ya que se colocó un resultado esperado incorrecto de forma intencional.

Evidencia de la prueba fallida intencionalmente:



Pruebas de interfaz

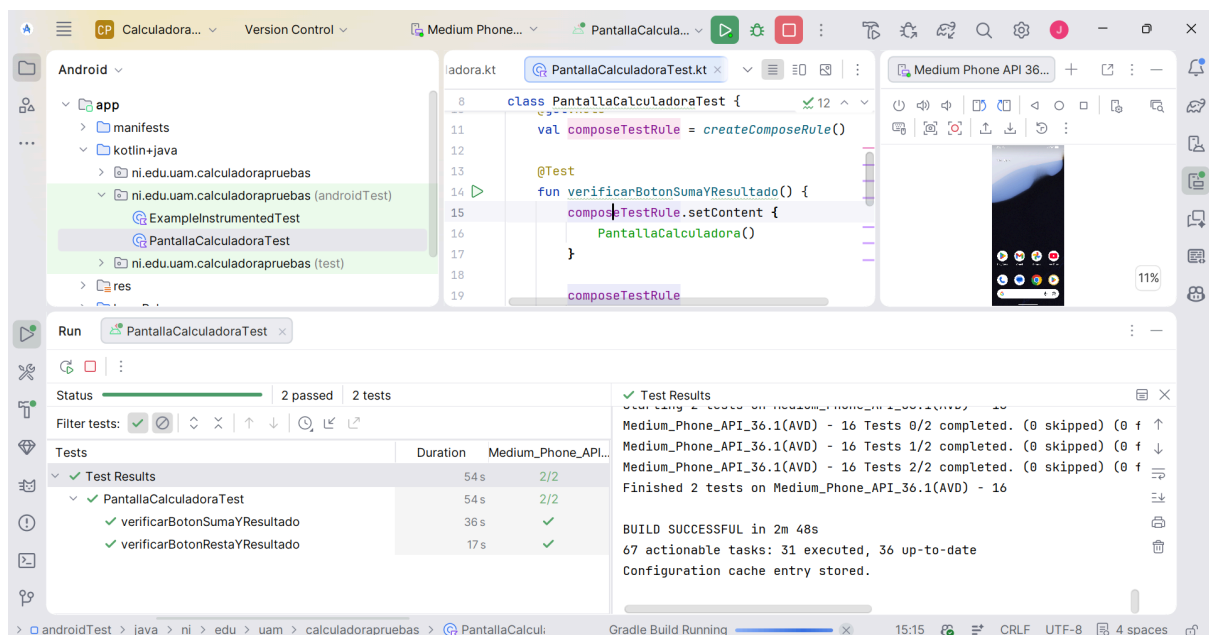
Las pruebas de interfaz fueron creadas en la carpeta androidTest del proyecto. A diferencia de las pruebas unitarias, estas pruebas sí requieren un emulador o un dispositivo Android, ya que verifican el comportamiento de los componentes visuales de la aplicación.

En esta práctica se creó la clase PantallaCalculadoraTest. La primera prueba de interfaz localiza el botón Calcular, realiza un clic sobre él y luego verifica que el texto de resultado muestre el valor 8.

Como parte de las actividades complementarias, se agregó una segunda prueba de interfaz para validar el botón Restar. Esta prueba localiza el botón Restar, realiza un clic sobre él y luego verifica que el texto de resultado muestre el valor 2.

Estas pruebas permiten comprobar que los botones funcionan correctamente y que la interfaz muestra el resultado esperado después de la interacción del usuario.

Evidencia de prueba de interfaz ejecutada correctamente:



Actividades complementarias realizadas

Durante la práctica se realizaron las actividades complementarias solicitadas:

1. Se verificó una operación de resta mediante una función restar y una prueba unitaria.
2. Se verificó una operación de multiplicación mediante una función multiplicar y una prueba unitaria.
3. Se verificó una operación de división mediante una función dividir y una prueba unitaria.
4. Se validó el comportamiento ante números negativos utilizando una prueba con valores negativos.
5. Se diseñó una prueba que falla intencionalmente, colocando como resultado esperado un valor incorrecto.
6. Se agregó un segundo botón a la interfaz, llamado Restar, y se creó una prueba de interfaz para validar su funcionamiento.

Estas actividades permitieron ampliar la funcionalidad inicial de la aplicación y comprobar diferentes escenarios mediante pruebas automatizadas.

Diferencia entre prueba unitaria y prueba de interfaz

Una prueba unitaria se enfoca en validar una parte específica del código, como una función o una clase. En este caso, las pruebas unitarias validaron los métodos de la clase Calculadora.

Estas pruebas no necesitan abrir la aplicación ni utilizar un emulador, porque solo comprueban la lógica interna.

Una prueba de interfaz, en cambio, valida el comportamiento visual e interactivo de la aplicación. En esta práctica, las pruebas de interfaz comprobaron que al presionar un botón en

pantalla, el resultado mostrado fuera el correcto. Estas pruebas sí requieren un emulador o dispositivo, ya que interactúan con la pantalla de la aplicación.

Preguntas de reflexión

1. ¿Cuál es la finalidad de las pruebas unitarias en el desarrollo de aplicaciones móviles?

La finalidad de las pruebas unitarias es comprobar que una parte específica del código funciona correctamente de forma independiente. En una aplicación móvil, estas pruebas ayudan a validar la lógica de negocio, como cálculos, validaciones o procesos internos, sin necesidad de ejecutar toda la aplicación. Esto permite detectar errores de manera rápida y facilita el mantenimiento del código.

2. ¿Qué ventajas ofrece Jetpack Compose para realizar pruebas de interfaz?

Jetpack Compose ofrece ventajas importantes para las pruebas de interfaz porque permite identificar y manipular componentes de forma sencilla mediante textos, etiquetas y nodos. Por ejemplo, se puede buscar un botón por su texto, realizar un clic automáticamente y verificar si un componente muestra el resultado esperado. Además, el uso de `testTag` facilita localizar elementos específicos dentro de la pantalla.

3. ¿Qué problemas pueden detectarse mediante pruebas automatizadas?

Mediante pruebas automatizadas se pueden detectar errores en cálculos, resultados incorrectos, fallos en botones, problemas al mostrar información en pantalla, errores de validación y comportamientos inesperados de la aplicación. También permiten identificar si una modificación en el código afecta una función que anteriormente funcionaba correctamente.

4. ¿Por qué es importante ejecutar pruebas antes de publicar una aplicación?

Es importante ejecutar pruebas antes de publicar una aplicación porque ayudan a asegurar que las funciones principales trabajan correctamente y que la interfaz responde como se espera. Esto reduce la posibilidad de entregar una aplicación con errores, mejora la experiencia del usuario y aumenta la confiabilidad del software.

5. ¿Qué diferencia existe entre probar la lógica de negocio y probar la interfaz gráfica?

Probar la lógica de negocio significa validar las funciones internas de la aplicación, como operaciones matemáticas o reglas de funcionamiento. En cambio, probar la interfaz gráfica significa verificar que los elementos visuales, como botones y textos, funcionen correctamente cuando el usuario interactúa con ellos. En esta práctica, la clase Calculadora representa la lógica de negocio, mientras que los botones y el texto de resultado representan la interfaz gráfica.

Conclusión

En esta práctica se logró desarrollar una aplicación Android básica utilizando Kotlin y Jetpack Compose, integrando pruebas unitarias y pruebas de interfaz. Las pruebas unitarias permitieron comprobar que las operaciones matemáticas de la clase Calculadora funcionaran correctamente, mientras que las pruebas de interfaz permitieron validar el comportamiento de los botones y la visualización del resultado en pantalla.

Además, mediante las actividades complementarias se ampliaron las funciones de la calculadora, se agregaron nuevos escenarios de prueba y se creó una prueba fallida intencionalmente para comprender cómo se detectan errores en los resultados esperados.

La actividad permitió comprender la importancia de las pruebas automatizadas en el desarrollo de software, ya que ayudan a mejorar la calidad de las aplicaciones, detectar errores a tiempo y asegurar un funcionamiento más confiable antes de entregar o publicar un proyecto.

Anexos

Link de Github:

https://github.com/JulioMS-DEV/Pruebas_Unitarias.git